Abstract

Graph neural networks (GNNs) are a promising class of machine learning algorithms for reasoning over structured relational data, making them a natural candidate for systems engineering problems involving requirement traceability and outlier detection. This work evaluates five GNN architectures (GCN, GAT, GATv2, GraphSAGE, and RGCN) on a synthetic systems engineering knowledge graph comprising requirements, functions, and components connected by hierarchical, allocation, and satisfaction relationships. Three levels of outlier complexity are defined and used to assess whether GNNs can detect invalid relationships and propose corrective edges to repair the graph. Performance is evaluated through extensive hyperparameter tuning and multiple metrics including precision, recall, and F1 score.

Introduction and motivation

The goal of the project is to explore the Graph Neural Network architectures' capabilities in detecting patterns and outliers in systems engineering models. Systems Engineering (in particular Model Based Systems Engineering) is a fairly young discipline focusing on managing complexity in engineering projects. A typical systems engineering model breaks down a product architecture into System Requirements, System Functions and Logical and Physical Components. Each one of those "domains" is decomposed through tiers representing decreasing levels of abstraction. The critical aspect of Systems Engineering is traceability consisting in the allocation of requirements at different levels of the architecture and ensuring that those requirements are satisfied. A requirement that is missed (either left unallocated or unsatisfied) can in some cases lead to catastrophic accidents leading to the loss of human life. For that reason, robust identification of gaps in traceability can indeed be a matter of life and death.

A classic example of a missed requirement leading to a catastrophic accident is the missed allocation of fatigue load stress testing on the De Havilland Comet fuselage leading to multiple accidents in the 50s and killing dozens of people (Withey, 2019). The square windows placed in the fuselage caused cracks in their corners due to fatigue following multiple pressurization and depressurization cycles.

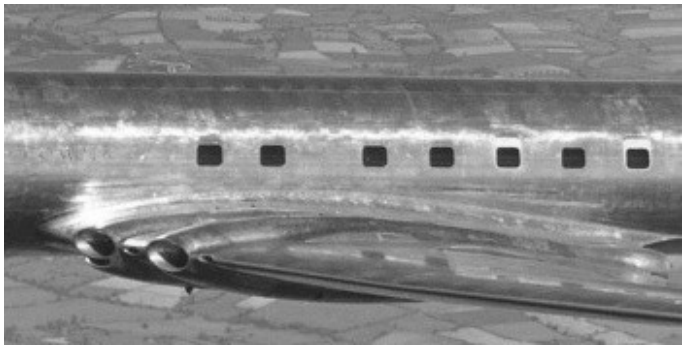

Figure 1 - De Havilland Comet's square windows

Technical background

Given that aerospace data is often proprietary, we had to resort to synthetic data. The first portion of the project consists in the generation of three sets of graphs where the outlier detection grows in complexity:

- Direct satisfaction of allocated requirement – an outlier in this case is any allocate or satisfy relationship that does not coexist with a complementary relationship going the opposite way.

Figure 2 - Simple outliers. Direct satisfaction of allocated requirement

- Decomposition of the allocation target – an outlier in this case is any allocate that does not coexist with a set of decompose and satisfy link whose chain completes the loop or a satisfy link that does not participate in such a loop.

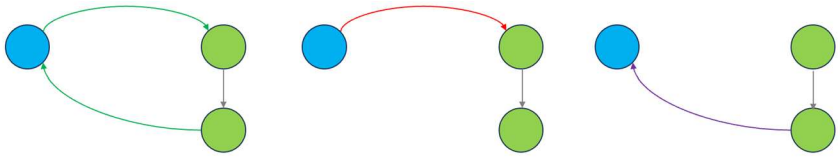

Figure 3 - Standard outliers. Decomposition of the allocation target

- Satisfying the sum of decomposed requirements – an outlier in this case is any allocate relationship where the set of decomposed requirements is not fully satisfied or a satisfy relationship that is redundant.

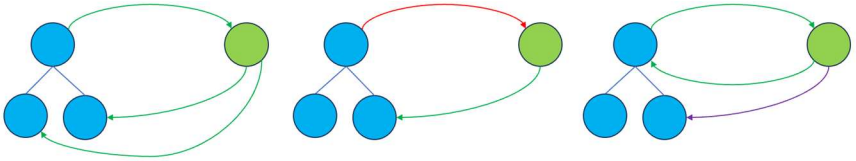

Figure 4 - Complex outliers. Satisfying the sum of decomposed requirements

The synthetic models we generated are sets of graphs where each domain is composed of 50 elements in a hierarchy of 3 tiers for requirements and 4 tiers for functions and components. The two plots below show a simple example with only 6 elements in each domain as well as a simplified clustered representation of a full graph representative of the data that was used for training the models.

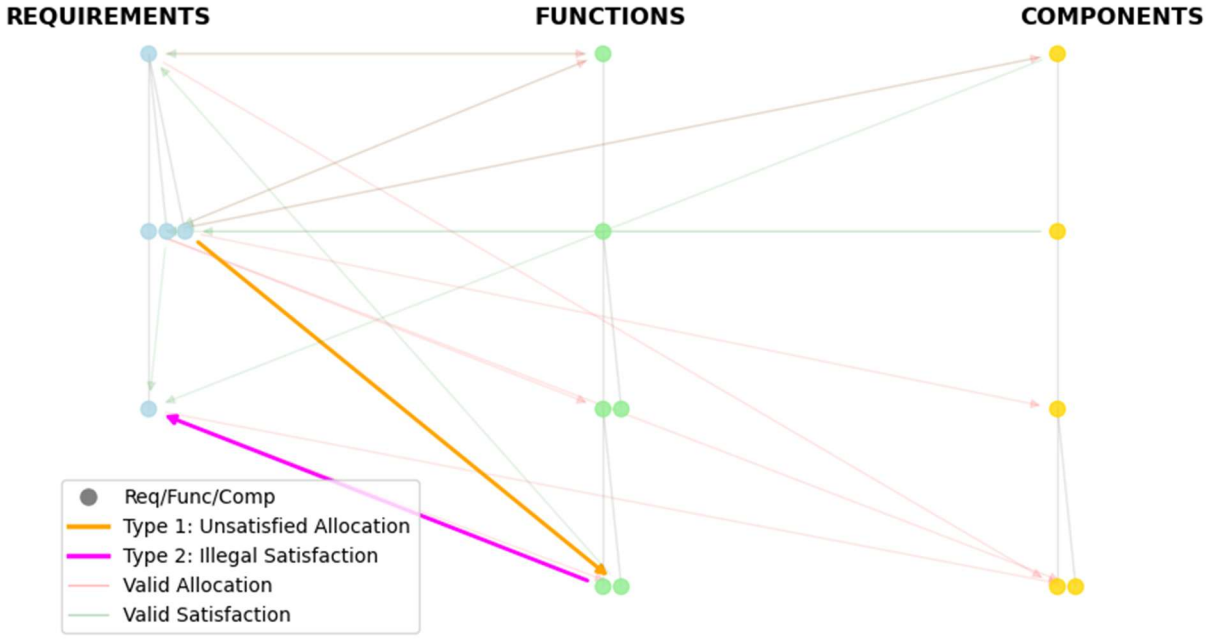

Figure 5 – Simple Systems Engineering graph.

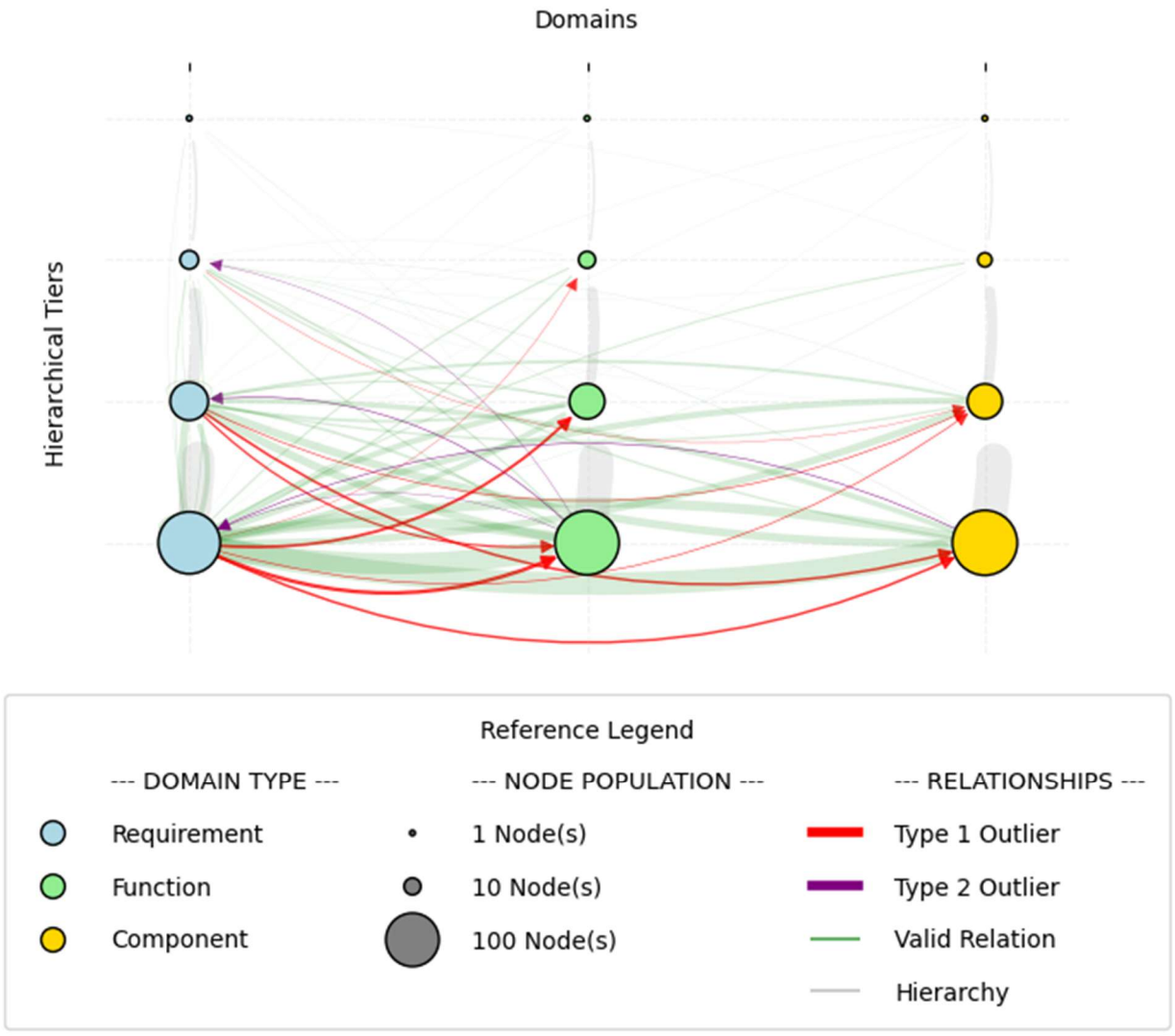

Figure 6 – Simplified clustered representation of full graph used for training.

Our project will explore the impact of the presence of those outliers on the performance of different GNN architectures. Several papers were published in recent years on the use of GNNs for detecting and completing missing relationships in Systems Engineering models. One such paper that was particularly well aligned with the idea for our project was recently published by the International Council on Systems Engineering (INCOSE) (Karagoz et al., 2025).

Methodology

Data Generation and Refinement

The Python library NetworkX was used to construct the synthetic systems engineering graphs as directed graphs, with three node types (requirements, functions, and components). Each tree was grown by first forcing a path to the target depth to guarantee the hierarchy reached the desired number of tiers, then randomly attaching remaining nodes under existing parents until the target size was reached. The three domain trees were then merged into a single graph, and allocation and satisfaction edges were randomly assigned between requirements and function/component nodes according to the outlier rules for each complexity level.

The synthetic graph was stored as a NetworkX directed graph where each node carried a dictionary of attributes — type and hierarchical level — and each edge carried a relation type and a binary outlier flag. This was converted to PyTorch Geometric format using a custom `convert_to_pyg_data` function, which mapped nodes to rows in a feature matrix with one-hot encoded types and normalized level, encoded edges as an index tensor, and assigned each edge a binary label for use as ground truth during training.

General GNN Methodology

GNNs are unique from other machine learning applications in that they act on completely differently structured datasets. The data consists of edges and nodes. An additional challenge is that the same graph can be expressed as many different adjacency matrices depending on node ordering — GNNs are designed to be permutation invariant (Sanchez-Lengeling et al., 2021) ,meaning the output is consistent regardless of how nodes are arranged.

The architectures evaluated include GCN, GAT, GATv2, GraphSAGE, and RGCN. In this implementation, each architecture uses the same loss function and optimizer. The loss function is binary cross-entropy with logits (BCE-with-logits), which is appropriate here because each edge has a binary classification: either outlier or valid. There is a strong imbalance between legitimate and outlier edges — roughly 95–5 for Type 2 and 90–10 for Type 1 — meaning a model could achieve a low loss simply by labeling most edges valid. To counteract this, the loss applies a `pos_weight` term that scales up the penalty for outlier edges, so the model is punished more heavily for missing a true outlier than for misclassifying a valid edge. The value of `pos_weight` is calculated during training as the ratio of valid edges to outlier edges. For example, if there are 1,500 valid edges and 100 outliers, `pos_weight` is 15 — meaning each outlier the model misclassifies contributes 15 times as much to the loss as a misclassified valid edge. The optimizer is Adam with a learning rate of 0.01.

GCN

All of the GNNs examined in this project share a similar pipeline — and indeed a similar structure to other neural network architectures — those being 1) a multilayer GNN encoder, 2) edge lookup, 3) MLP edge classifier, 4) loss function, and 5) optimizer. The differences between the architectures are mainly in the first three steps. Figure 6 below illustrates the first three steps for the simplest GNN architecture, the Graph Convolutional Network (GCN).

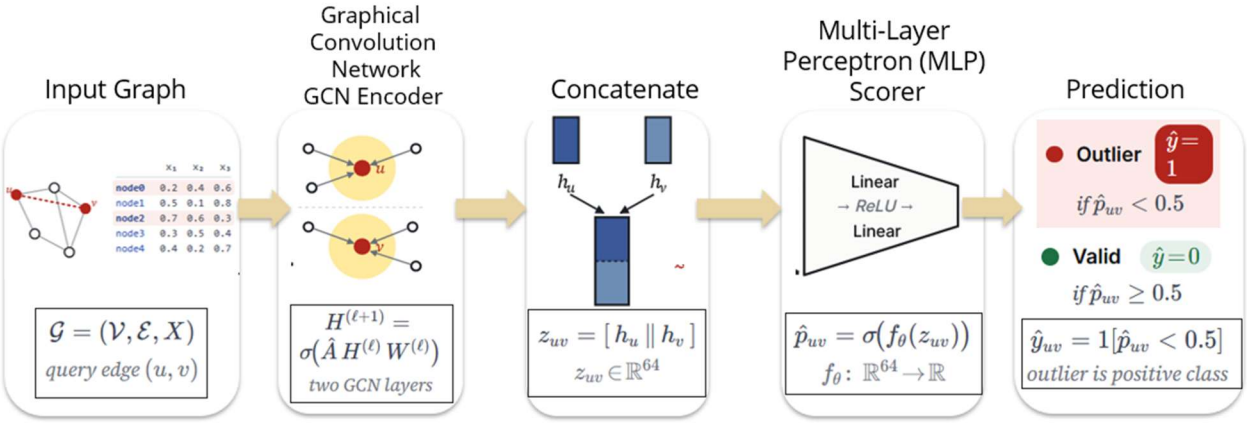

Figure 7 - Edge prediction pipeline of a GCN

An important aspect of the GCN (Kipf & Welling, 2017) is that the data must be preformatted correctly — this is a key difference from conventional neural networks. The matrix, referred to as X, represents a node on each row. Each column is representative of various features, and these are typically one-hot encoded to describe node type. The

order of nodes in X must be maintained for the rest of the pipeline. This order is used to create an adjacency matrix, which describes connections between the different nodes.

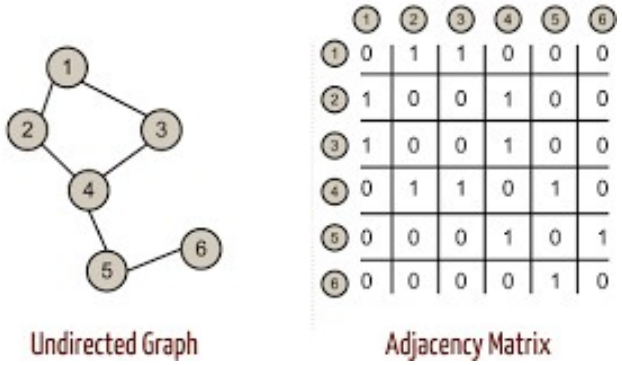

Figure 8 - Adjacency Matrix Example with Simple Undirected Graph

Along with the input matrix X and the adjacency matrix A, the first layer of the encoder uses a trainable weight matrix W, tuned during the training phase. The first layer produces the first embedding values h

$$H = \sigma(\hat{A}XW) \qquad \textit{Equation 1}$$

where H is called a node embedding. As seen in Equation 1, it accounts for neighboring connections using the adjacency matrix. A key limitation of this is that all edges are treated equally — the adjacency matrix assigns a value of 1 for an edge or 0 otherwise. It is binary and unable to weight edges differently.

The second layer operates the same as the first, but now effectively considers nodes that are two hops away. A two-layer encoder was implemented to start, but more layers could be added to consider further-removed nodes.

After the encoder has produced the final embedding matrix H, individual node embeddings are extracted and concatenated pairwise for each query edge. Consider two nodes, u and v — the embeddings for u and v are concatenated as [h_u ‖ h_v]. H maintains the same row ordering as X, so each row of the matrix corresponds to a node in the same order. At this point in the pipeline, [h_u ‖ h_v] is representative of the edge between nodes u and v. This is passed to the MLP, which consists of two linear layers, one ReLU in between, and one output neuron. A sigmoid function converts this output to a probability, and edges with a predicted probability above 0.5 are classified as outliers. This edge classification formulation is consistent across all five architectures — only the encoder that produces H varies between them.

GATv2

An important limitation of the GCN, as noted above, is that the edges are all weighted equally. In practice, certain neighbors in a graph will carry more relevant information than others. The Graph Attention Network v2 (GATv2) (Brody et al., 2022) incorporates this into its approach. For each directed edge (i → j), the model computes a raw attention score e_ij — a scalar that represents how important neighbor j is when updating node i's embedding. In GATv2, this score is computed as: $e_{ij} = a^T \cdot \text{LeakyReLU}(W[h_i \| h_j])$, where W is a learned weight matrix and a is a learned attention vector. e_ij is then normalized across all of node i's neighbors using softmax to produce α_ij — the final attention weight. α_ij values for all neighbors of a given node sum to 1, making them interpretable as proportional contributions.

The full pipeline follows these steps: (1) GATv2 Layer 1 — attention scores are computed and neighbors are aggregated, producing node embeddings H1; (2) GATv2 Layer 2 — a second round of attention-weighted aggregation compresses embeddings to H2; (3) edge lookup — for each edge (u → v), the embeddings h_u and h_v are

concatenated into an edge vector; (4) MLP classifier — the edge vector is passed through two linear layers with a ReLU activation, producing a single logit per edge; (5) prediction — a sigmoid function converts the logit to a probability, thresholded at 0.5 to classify the edge as outlier or valid. The probability is then output to the loss function and optimizer as before.

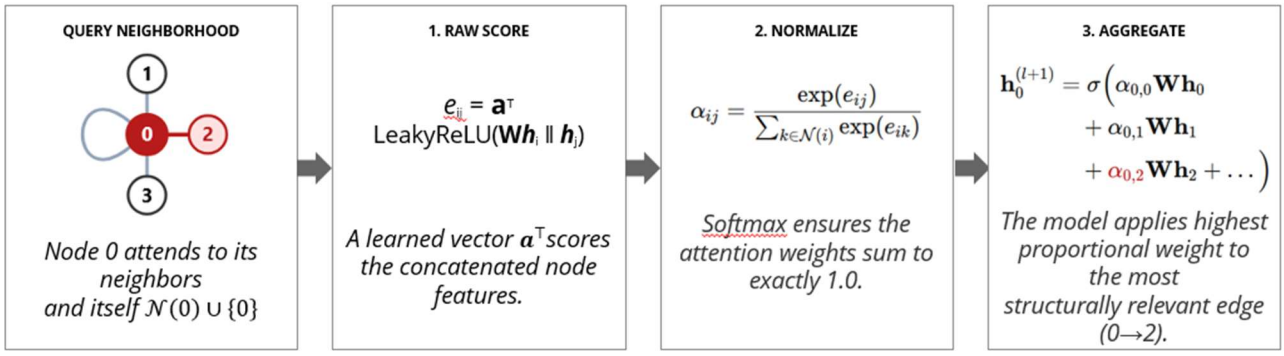

Figure 9 - GATv2 attention mechanism: raw score computation, softmax normalization, and weighted aggregation

The output for each edge is a binary label: 1 = outlier, 0 = valid. Further detail on the edge classification setup will be provided in the final report. Model performance has been evaluated using precision (of the edges flagged as outliers, how many actually are), recall (of all true outliers, how many were caught), and F1-score (the harmonic mean of the two), reported separately for Type 1 and Type 2 outliers.

GAT

The Graph Attention Network (GAT), introduced by Veličković et al. (2018), uses the same attention-weighted aggregation concept as GATv2 but with an earlier scoring formulation: the weight matrix W is applied to each node's embedding separately before concatenation, rather than jointly. This makes GAT's attention weights less context-sensitive than GATv2's — a distinction that motivates comparing the two architectures directly on the same task.

GraphSAGE

GraphSAGE (Graph Sample and Aggregate), introduced by Hamilton et al. (2017), departs from GCN by aggregating a sampled subset of each node's neighbors rather than the full neighborhood, making it well-suited to large graphs where processing all neighbors at once is computationally expensive. Like GCN, GraphSAGE does not incorporate edge type information during message passing, so it must infer outlier patterns from node embeddings and graph structure alone.

RGCN

The Relational Graph Convolutional Network (RGCN), introduced by Schlichtkrull et al. (2018), extends the GCN by learning a separate weight matrix for each edge type — in this case hierarchy, allocation, and satisfaction — allowing it to treat structurally distinct relationships differently during aggregation. This makes RGCN a natural fit for this problem, since outlier status is inherently relational: validity depends on whether the right combination of edge types is present between a requirement and its allocated function or component.

Graph repair

On top of the outlier detection, we also implemented a simple graph repair algorithm. The approach taken is to perform two sequential tasks sharing the same GNN backbone. In the detection task, the GNN runs message passing over the full graph so that each node accumulates structural context from its neighborhood. The detection head

scores each allocation edge for the probability of being an outlier. Once an outlier is detected, a repair pass re-runs the GNN with satisfies edges removed and a repair head scores each candidate node in the target's subtree to identify which one should receive the new satisfies edge.

Results

All five architectures were evaluated on a synthetic systems engineering graph consisting of 500 requirement nodes, 500 function nodes, and 500 component nodes. Each model was trained for 1,001 epochs using the Adam optimizer with a learning rate of 0.01. Figure 9 shows the detection accuracy for Type 1 and Type 2 outliers across all five architectures. Type 1 outliers — unsatisfied allocations — were generally harder to detect, with the best accuracy reaching 83.1%. Type 2 outliers — illegal satisfactions — were more consistently detected, with RGCN achieving the highest accuracy at 98.1%.

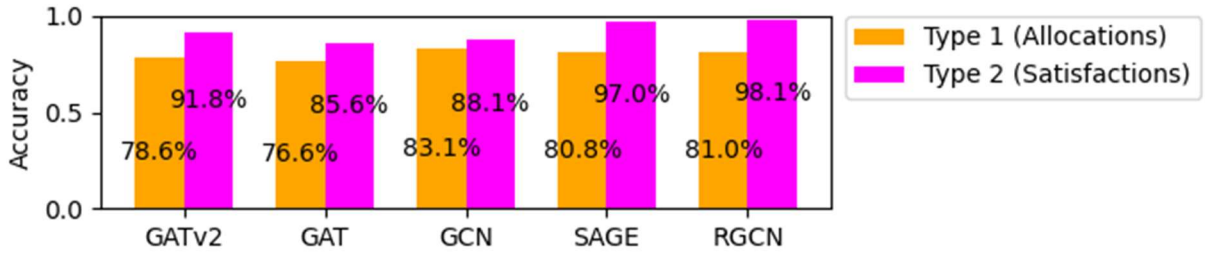

Figure 10 - Accuracy of different GNN architectures on the standard model

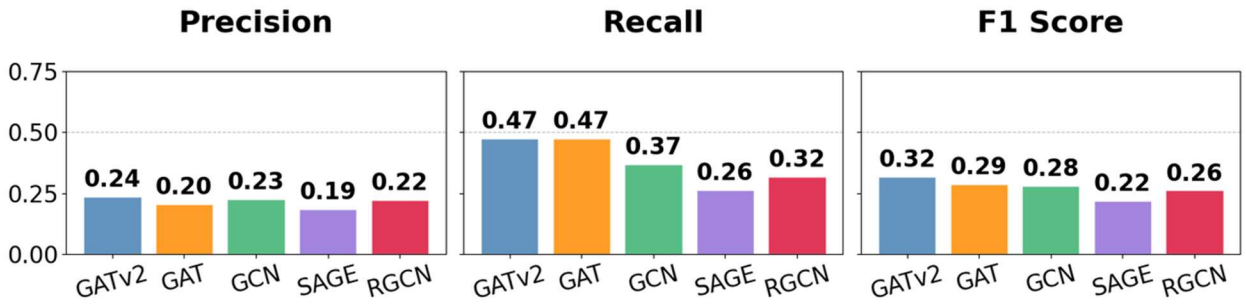

Figure 11: Precision, recall, and F1-score on held-out graphs; recall is the priority metric, since a missed outlier is an unflagged traceability gap.

Figure 10 reports precision, recall, and F1 score for all five architectures. The accuracy values in Figure 9 can be misleading given the class imbalance in the dataset — with a 10% injection rate for Type 1 outliers and 5% for Type 2, valid edges significantly outnumber outliers, meaning a model can achieve high accuracy while catching very few true outliers. Recall is therefore the more meaningful metric for this problem: a missed outlier is a traceability gap that goes unflagged, which is the failure the project is meant to prevent, so ideally recall would approach 1.0. The best recall observed was 0.47 (GAT and GATv2), precision peaked at 0.24 (GATv2), and GATv2 gave the best overall balance with an F1 of 0.32 — leaving clear room to improve. Two findings were unexpected. First, GAT and GATv2 performed identically on recall (**0.47**); we had expected the more dynamic attention in GATv2 to provide a distinct advantage on the training data, but it did not. Second, GCN outperformed RGCN (F1 **0.28 vs. 0.26**), which is surprising since GCN is blind to edge type while RGCN trains on the different edge types directly. It is also worth noting that this evaluation was performed on the same graph used for training — the relatively modest performance even on the training graph suggests the models have not fully converged on the outlier detection task. To investigate generalization further, an additional graph was generated to assess performance on unseen data.

Figure 11 shows the performance metrics of the trained models on 10 unseen graphs with the same sizes (500 req's, 500 components, 500 functions). These graphs were formed in the same method as the original training graph. With the amount of random edges, the model would be expected to perform just as well or perhaps worse than the original training graph. The unseen graphs saw similar performance across the three metrics. Based on this comparison, it would seem that the model wasn't overtrained on the data, and similar performances could be found on unseen models.

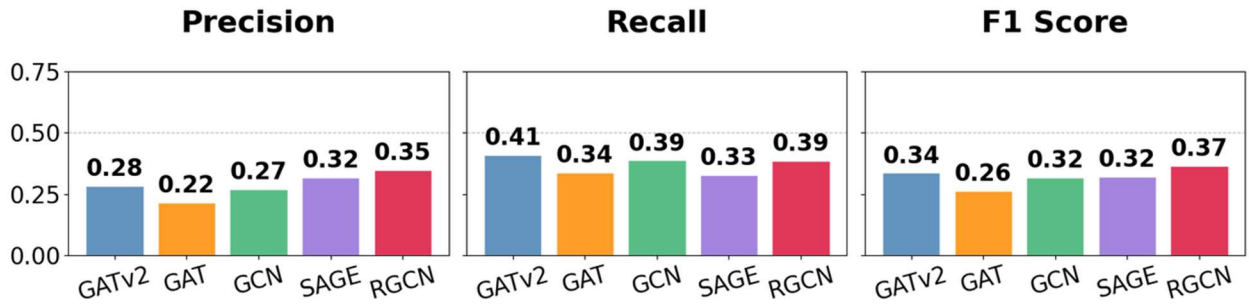

Figure 12 Unseen Graphs -- GAT is the best observed Architecture for Recall

The models examined thus far do not provide much value for the purpose of identifying and flagging potential missing links. In order to improve the recall score for the different models, a higher penalty must be enacted on the models during training. This can be accomplished by scaling up the effect of the `pos_weight` by various scaling factors into the loss function. Various scaling factors were applied to pos_weight for different training runs of GATv2 and this data from this sweep can be seen below in Figure 12. The data demonstrates that increasing the effect of the `pos_weight` into the loss function will improve the recall dramatically, albeit at the cost of the precision.

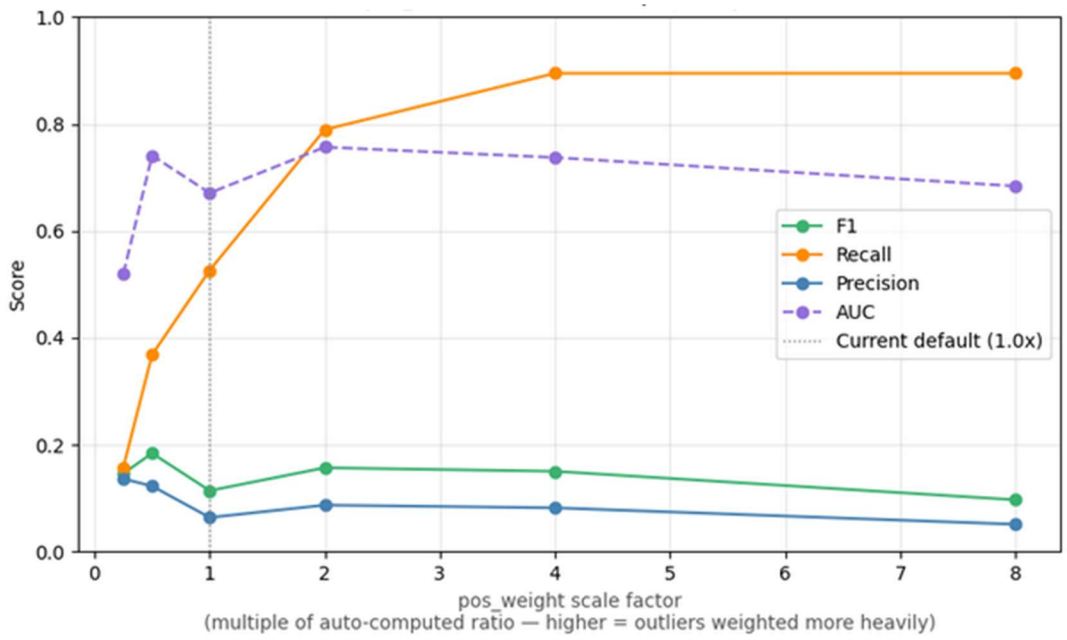

Figure 13: GATv2 Benefitted From Higher pos_weight Scaling Factors when Evaluating Recall

The precision score appears to be optimized at a `pos_weight` scale of 4 for the GATv2. The other architectures were run as well with a `pos_weight` scaling factor of 4.0, but the results demonstrated that GATv2 was the only

architecture to drastically improve from the higher `pos_weight` scaling factor. Importantly, it must be noted that the precision is very poor for all architectures under, the GATv2 architecture was only able to achieve 0.08.

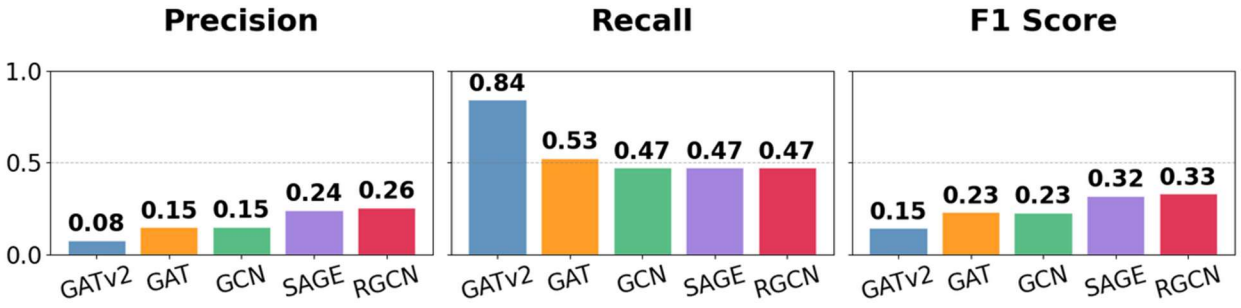

Figure 14: Precision and Recall Scores Did Not Drastically Improve for pos_weight scaling factor of 4.0.

We also attempted to use the same GNN architectures to propose repairs to the graph. We achieved average results at fixing the outliers when the model made a correct determination of an edge being an outlier, but given the poor performance of the model (in particular the recall metric) in most cases only half of the outliers were identified and, of that half, only half were correctly repaired. We also see that, in some cases, the models repair a large amount of relationships that are not broken.

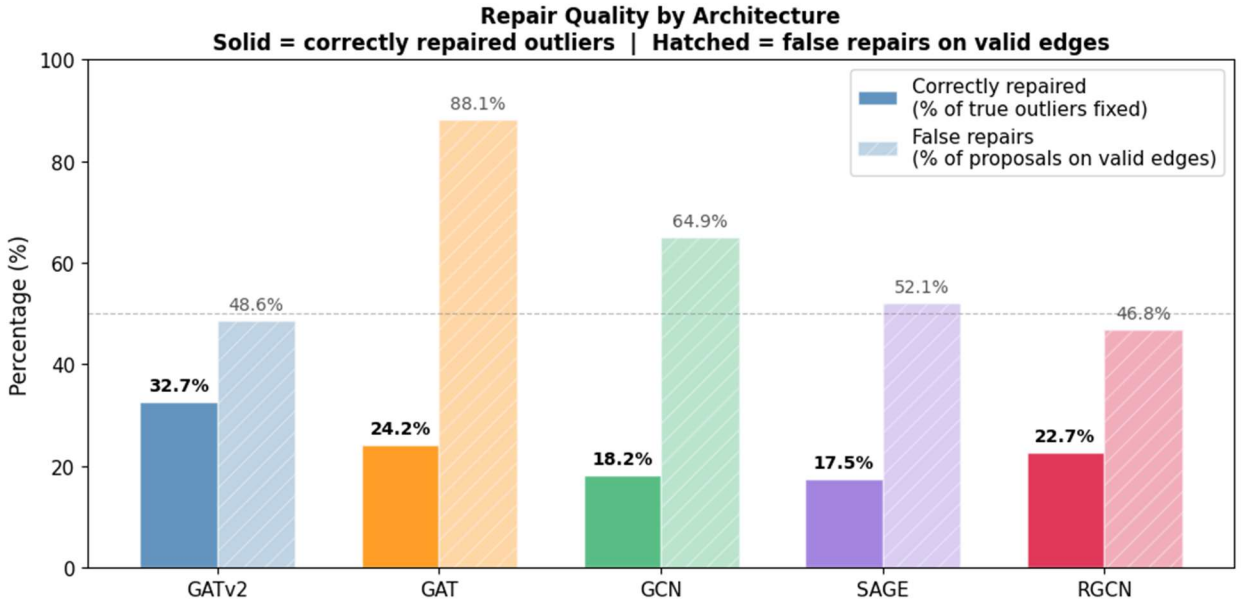

Figure 15: Repair Quality by Architecture

Conclusion

We must admit that despite endless efforts to tune the parameters of the models, we were not able to achieve results that sufficiently outperform a random classifier at outlier detection. The graph repair component, which proposed new satisfies edges for detected outliers, was similarly limited by the weak detection stage feeding into it rather than by the repair mechanism itself. What we think may be contributing to the poor results is that the outlier signal requires joint reasoning across domain boundaries (requirements, functions and components), and standard message passing tends to dilute that cross-domain signal by aggregating over all neighbor types indiscriminately. In order to improve the results, additional exploratory work would need to be performed on multi-layer or more

heterogeneous GNNs. Other ways of identifying outliers in the training data may also help. However, consistently poor F1 metrics across five architectures, three rule complexities, and multiple hyperparameter configurations, seems to indicate that GNNs may not be the ideal solution for the chosen problem and rule or query-based methods may be a better fit. What the results also tell us is that the problem of pattern discovery in large and deeply nested graph structures is not an easy one to solve (something we already know from professional experience).

Summary

Graph neural networks, a class of machine learning algorithms that learn node representations by iteratively aggregating information from neighboring nodes, were explored in the context of a systems engineering problem involving requirement traceability and outlier relationship detection. Synthetic data representing different levels of complexity requiring deeper graph explorations and information aggregation was generated and used to evaluate five different GNN architectures for their ability to detect outliers and propose edge additions to repair them. Several parameters were explored to enable the models to correctly separate valid relationships from invalid ones, and multiple metrics such as precision, recall and F1 score were used to evaluate performance. Ultimately, the results indicate that the models do not sufficiently outperform random classifiers and that further work is needed to either change the outlier encoding in the training data, find alternative GNN architectures, or reformulate the problem.

Link to Code Repository

https://github.com/tcheb/ENGR521A/blob/main/Synth_SE_data.ipynb

CRediT Statement

Alek: Conceptualization, Data curation, Formal analysis, Investigation, Methodology (partial), Resources, Software (data generation, graph repair), Visualization, Writing – original draft (Introduction, Technical Background, Results (repair), Summary), Writing – review & editing

Zack: Formal analysis, Investigation, Methodology (partial), Software (GNN implementation, hyper-parameter exploration, result collection), Visualization, Writing – original draft (Methodology (GNN), Results (detection)), Writing – review & editing

Symbol and abbreviation glossary

Symbol	Definition	Symbol	Definition
$\mathcal{G}$	Graph, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X)$	$\mathcal{V}$	Node set (requirements, functions, components)
$\mathcal{E}$	Edge set (allocate, satisfy, decompose)	X	Node feature matrix (one-hot node types)
A	Adjacency matrix	$\hat{A}$	Normalized adjacency matrix
W	Learnable layer weight matrix	ℓ	Layer index
H	Node embedding matrix (layer output)	h_i, h_u, h_v	Single-node embedding vector, $\in \mathbb{R}^{32}$
[h_u ‖ h_v]	Concatenated edge vector	z_uv	Edge vector for (u, v), $\in \mathbb{R}^{64}$
f_θ	MLP edge classifier, $\mathbb{R}^{64} \rightarrow \mathbb{R}$	θ	MLP learnable parameters
p̂_uv	Predicted outlier probability for (u, v)	ŷ_uv	Predicted label (1 = outlier, 0 = valid)
e_ij	Raw attention score, edge i → j (GATv2)	a	Learned attention vector (GATv2)
α_ij	Normalized attention weight (Σ = 1)	$\mathcal{N}(i)$	Neighborhood of node i (with self-loop)
σ	Activation (ReLU encoder; sigmoid output)	pos_weight	Loss weight up-scaling outlier edges

Table 1 **– Symbol glossary**

Abbr	Definition	Abbr	Definition	Abbr	Definition
MBSE	Model-Based Systems Engineering	GNN	Graph Neural Network	GCN	Graph Convolutional Network
GAT	Graph Attention Network	GATv2	Graph Attention Network v2	GraphSAGE	Graph Sample and Aggregate
RGCN	Relational GCN	MLP	Multilayer Perceptron	BCE	Binary Cross-Entropy
ReLU	Rectified Linear Unit	LeakyReLU	Leaky ReLU	Adam	Adaptive Moment Estimation
F1	F1-score (harmonic mean of P/R)	INCOSE	Int'l Council on Systems Engineering		

Table 2 **– abbreviation glossary**

Bibliography

E.Karagoz, O. J. (2025). Identification of Missing Knowledge in MBSE System Models Using Graph-Based Machine Learning. *INCOSE*, (pp. 133-149).

Withey, P. (2019, 01 24). *The Real Story of the Comet Disaster.* Retrieved from University of Birmingham: https://www.fzt.haw-hamburg.de/pers/Scholz/dglr/hh/text_2019_01_24_Comet.pdf

Sanchez-Lengeling, B., Reif, E., Pearce, A., & Wiltschko, A. B. (2021, September 2). A Gentle Introduction to Graph Neural Networks. Retrieved from Distill: https://distill.pub/2021/gnn-intro/

Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *International Conference on Learning Representations*. Retrieved from https://openreview.net/forum?id=SJU4ayYgl

Brody, S., Alon, U., & Yahav, E. (2022). How Attentive are Graph Attention Networks? *International Conference on Learning Representations*. Retrieved from https://openreview.net/forum?id=F72ximsx7C1

Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. *Advances in Neural Information Processing Systems*, 30. Retrieved from https://arxiv.org/abs/1706.02216

Schlichtkrull, M., Kipf, T. N., Bloem, P., van den Berg, R., Titov, I., & Welling, M. (2018). Modeling Relational Data with Graph Convolutional Networks. In *The Semantic Web: ESWC 2018* (pp. 593–607). Springer.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. *International Conference on Learning Representations*. Retrieved from https://arxiv.org/abs/1710.10903